МОСКОВСКИЙ АВИАЦИОННЫЙ ИНСТИТУТ
(НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ)

Институт №8 «Компьютерные науки и прикладная математика»
Кафедра 806 «Вычислительная математика и программирование»

Лабораторная работа №1
по курсу «Компьютерная графика»

Основы 3D графики

Выполнил: Ле Шон Лыонг
Группа: М8О-309БВ-24
Преподаватель: В. Д. Бахарев

Москва, 2026

Условие

Цель работы — построить простой трёхмерный объект, изучить его проецирование на плоскость и аффинные преобразования. Реализация должна использовать Vulkan, GLFW и ImGUI и работать в реальном времени.

Вариант №1 — куб.

Выполнены основной вариант и первые пять дополнительных заданий. Реализованы переключение проекции; управление переносом, вращением и масштабом; движение и вращение по сложной траектории с паузой и изменяемыми параметрами; изменение цвета; процедурная окраска вершин с умножением на выбранный оттенок.

Метод решения

Куб задан восемью вершинами в углах области $[-0{,}5;\ 0{,}5]^3$. Каждая из шести граней разбита на два треугольника. Используются 12 треугольников и 36 индексов типа uint16_t. Вершины хранятся в vertex buffer, индексы — в index buffer.

При использовании векторов-столбцов матрица модели имеет вид $M = T \cdot Rz \cdot Ry \cdot Rx \cdot S$. Сначала выполняется масштабирование, затем вращение вокруг осей X, Y и Z, после чего перенос. Параметры задаются в интерфейсе независимо для каждой оси.

Позиция вершины в вершинном шейдере вычисляется по формуле:

$$pclip = P \cdot V \cdot M \cdot (x, y, z, 1)^T$$

Матрица вида V строится функцией lookAtRH для фиксированной камеры. Перспективная проекция использует угол обзора FOV и отношение ширины области сцены к высоте. После деления на w дальние объекты выглядят меньше. В ортографической проекции размер не зависит от расстояния до камеры.

Ближняя плоскость равна 0,1, дальняя — 100. Используются функции GLM perspectiveRH_ZO и orthoRH_ZO с диапазоном глубины [0; 1], требуемым Vulkan. Знак элемента P[1][1] инвертируется для согласования направления оси Y с viewport.

Метод решения

Параметры объекта передаются через один uniform buffer и один VkDescriptorSet. Привязка binding 0 имеет тип VK_DESCRIPTOR_TYPE_UNIFORM_BUFFER. Буфер содержит mat4 MVP, vec4 tint и ivec4 options: 64 + 16 + 16 = 96 байт, что соответствует размещению std140. Наборы дескрипторов ImGui используются отдельно только для интерфейса.

Графический конвейер использует TRIANGLE_LIST. Вершинный шейдер применяет матрицу MVP и назначает цвет Cvertex = position + (0,5; 0,5; 0,5). Значения локальных координат превращаются в компоненты RGB от 0 до 1. Итоговый цвет — покомпонентное произведение Cvertex и Ctint. Растеризатор интерполирует цвет между вершинами, фрагментный шейдер выводит его в framebuffer. При отключении Local vertex colors получается однотонная окраска.

Для правильного перекрытия поверхностей включены запись глубины и depth test с VK_COMPARE_OP_LESS. Цветовой буфер очищается перед кадром, буфер глубины — значением 1. Viewport, scissor и aspect ratio учитывают текущий размер окна.

Анимация задаётся параметрической траекторией:

$$r(t) = (R \cos t,\ H \sin 2t,\ R \sin t)$$

При $R > 0$ и $H > 0$ траектория замкнута и не лежит в плоскости. Смещение прибавляется к ручной позиции, а вращение по трём осям — к ручным углам. Локальное время увеличивается на $\Delta t \cdot \text{speed}$. Пауза замораживает время, поэтому после продолжения нет скачка. В интерфейсе изменяются скорость, радиус R, амплитуда H и скорость вращения.

Метод prepare() ожидает fence предыдущего кадра до записи uniform buffer. После записи вызывается vmaFlushAllocation. Затем записываются команды рисования куба и интерфейса, кадр отправляется в очередь и представляется на экран. При завершении ожидается idle и освобождаются Vulkan-ресурсы.

Результаты

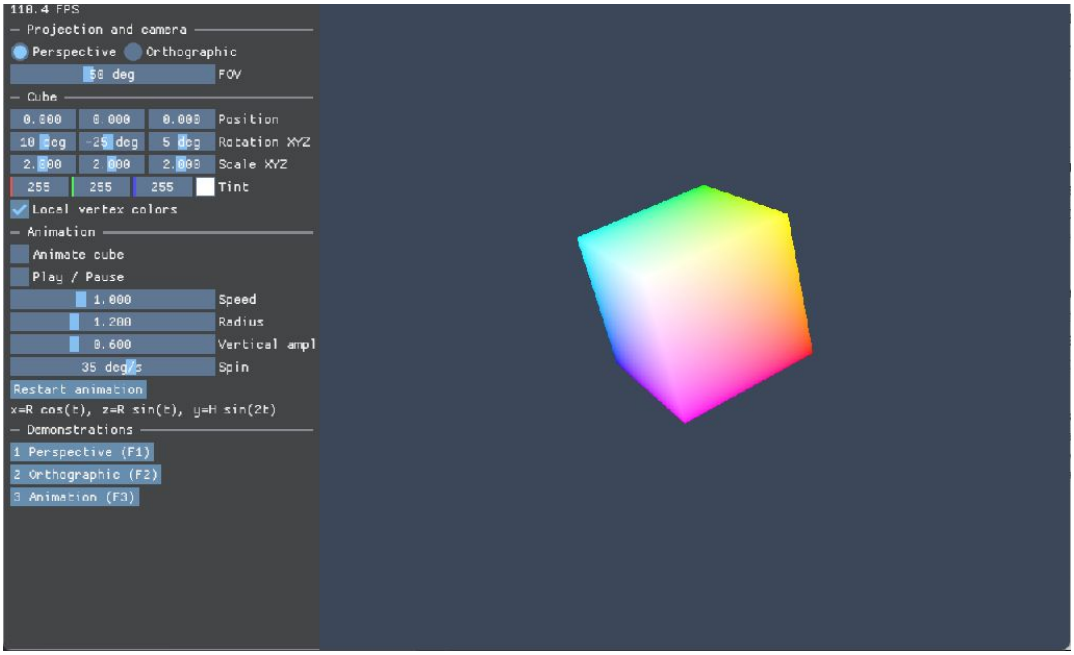


Рисунок 1 — Куб в перспективной проекции. Цвет каждой вершины определяется локальными координатами, между вершинами образуется градиент.

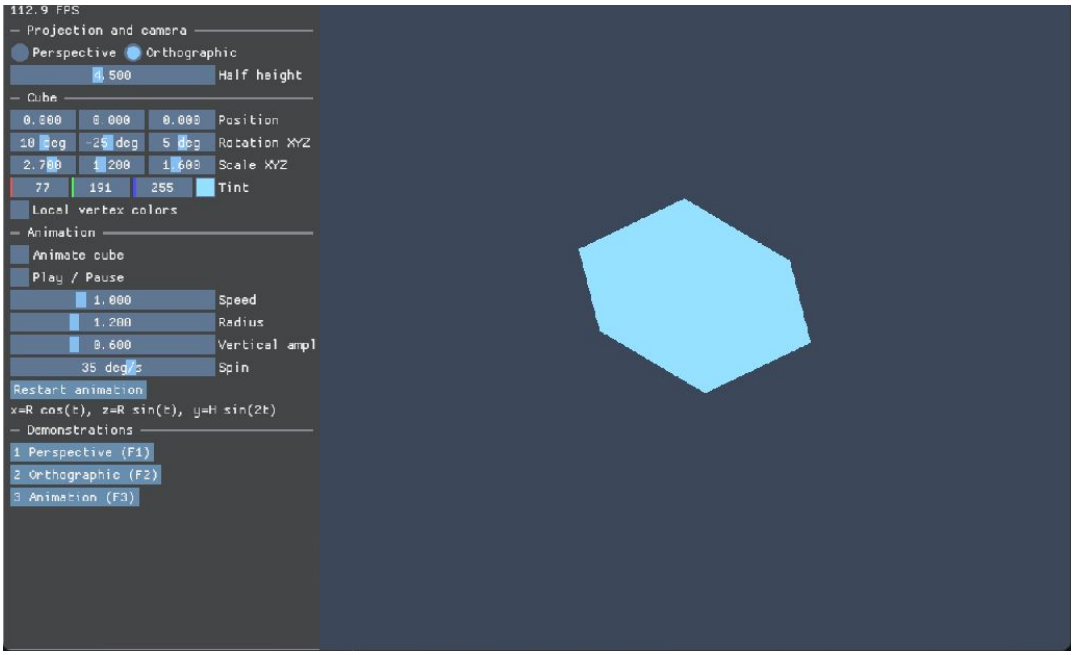


Рисунок 2 — Ортографическая проекция. Масштаб по осям различается; через Tint задан голубой цвет, процедурная окраска отключена.

Результаты

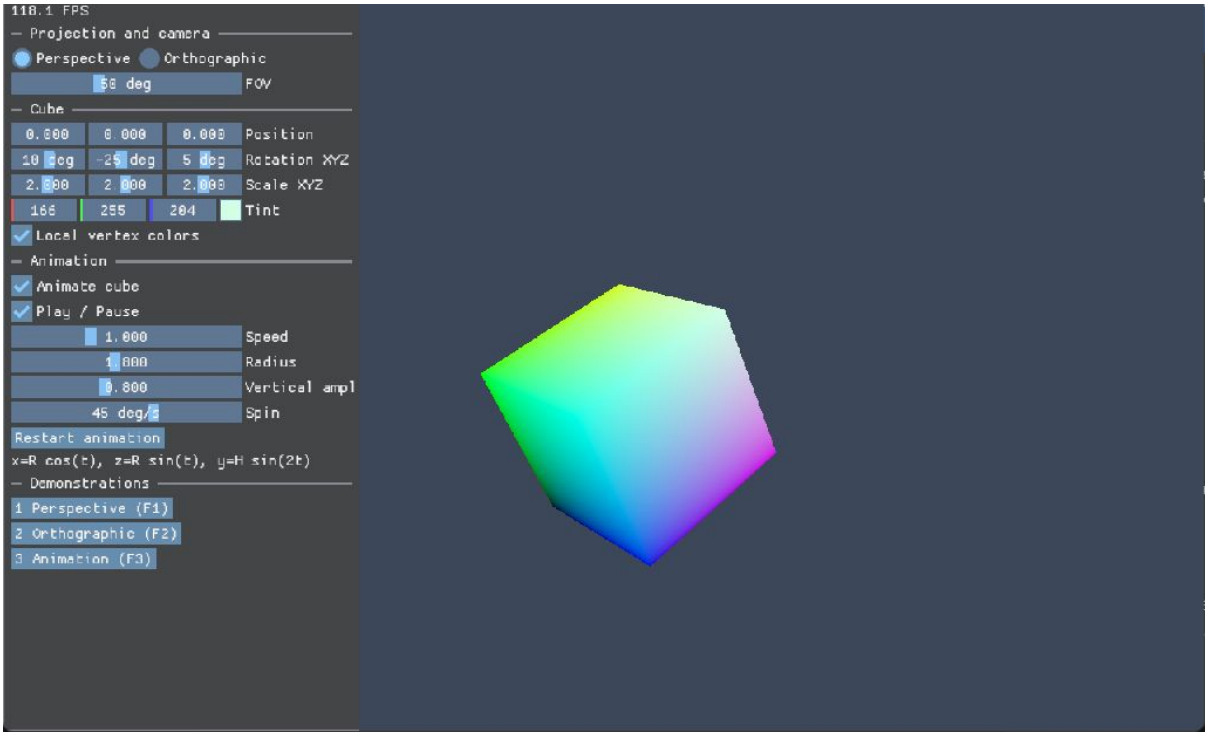


Рисунок 3 — Движение и вращение одного куба. Включены Animate cube и Play / Pause, радиус равен 1,8, вертикальная амплитуда — 0,8.

Выводы

Реализована интерактивная программа на C++20 и Vulkan для отображения куба. Выполнены основной вариант и первые пять дополнительных заданий: две проекции, аффинные преобразования, управляемая анимация, изменение оттенка и процедурные цвета вершин. Проверены запуск программы, переключение режимов, движение и остановка анимации.

Результат показывает различие перспективной и ортографической проекций, влияние порядка матриц на положение объекта и передачу параметров из интерфейса в шейдер через uniform buffer. Шестое дополнительное задание с несколькими объектами исключено в соответствии с выбранным объёмом работы.